

INFORME TECNICO

Red Neuronal Artificial para Clasificación

Dataset Iris

[bootcamp/Modulo8/RNA_clasificacion_Iris.ipynb at main · jlatorre-ing/bootcamp](https://github.com/jlatorre-ing/bootcamp/blob/main/Modulo8/RNA_clasificacion_Iris.ipynb)

Bootcamp Data Sciences Alkamy 2026

Integrantes:

Francisca Rojas

Carolina Mamani

José Tomas Latorre

1. Introducción

Este informe documenta el desarrollo y análisis de una Red Neuronal Artificial (RNA) para resolver un problema de clasificación multiclase utilizando el clásico dataset Iris. El trabajo abarca desde la preparación de los datos hasta la optimización de hiperparametros mediante técnicas sistemáticas de búsqueda.

El objetivo principal es demostrar el flujo completo de un proyecto de Machine Learning: carga de datos, preprocesamiento, construcción de la red, entrenamiento, evaluación y mejora del modelo mediante búsqueda de hiperparametros.

2. Dataset Iris

El dataset Iris es uno de los conjuntos de datos más conocidos en Machine Learning, introducido por el botánico Ronald Fisher en 1936. Contiene mediciones de 150 flores de tres especies diferentes.

Característica	Detalle
Total de muestras	150 flores
Numero de clases	3 (setosa, versicolor, virginica)
Numero de variables	4 (largo/ancho de sepalo y petalo)
Distribución de clases	50 muestras por clase (balanceado)
División entrenamiento/validación	80% / 20% (con stratify)

3. Arquitectura de la Red Neuronal

Se diseño una red neuronal secuencial con la siguiente estructura de capas:

Capa	Neuronas	Activación	Parámetros
Entrada (Input)	4	-	-
Ocultas 1 (Dense)	16	ReLU	80
Ocultas 2 (Dense)	8	ReLU	136
Salida (Dense)	3	Softmax	27
Total	-	-	243

Decisiones de diseño

- ReLU en capas ocultas: evita el problema del gradiente desvaneciente y es computacionalmente eficiente.
- Softmax en la salida: convierte los valores de salida en probabilidades que suman 1, ideal para clasificación multiclase.
- sparse_categorical_crossentropy: función de pérdida usada porque las etiquetas son enteros, no vectores one-hot.
- Optimizador Adam: ajusta automáticamente la tasa de aprendizaje durante el entrenamiento.

4. Entrenamiento del Modelo Base

El modelo base se entrenó en dos rondas consecutivas sin reiniciar los pesos aprendidos:

Ronda	Épocas	Batch Size	Accuracy Final (val)
Primera ronda	40	16	76.67%
Segunda ronda	50	16	93.33%
Total acumulado	90	16	93.33%

El modelo mostro una convergencia progresiva y estable. Las curvas de entrenamiento y validación bajaron juntas sin evidencia de sobreajuste significativo.

5. Análisis Visual de Errores del Modelo Base

Esta sección representa el aporte personal al notebook original. Se agrego una visualización de la matriz de confusión mediante un mapa de calor (heatmap) para facilitar la interpretación de los errores del modelo.

5.1 Matriz de confusión - Modelo Base

La matriz de confusión permite identificar exactamente que clases se están confundiendo entre si. A diferencia de la representación en texto plano del notebook original, el mapa de calor hace visualmente inmediata la interpretación.

5.2 Resultados del Modelo Base

Clase	Precisión	Recall	F1-Score	Support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
macro avg	0.93	0.93	0.93	30

5.3 Interpretación de los Resultados

- Setosa: perfectamente separable. El modelo clasifica las 10 muestras correctamente sin ningun error. Esto es esperado ya que setosa es linealmente separable de las otras dos clases.

- Versicolor: 1 error de clasificación. Una muestra fue confundida con virginica.
- Virginica: 1 error de clasificación. Una muestra fue confundida con versicolor.
- La confusión entre versicolor y virginica es un resultado clasico con este dataset, ya que ambas clases tienen características morfológicas similares y sus distribuciones se solapan en el espacio de características.

El accuracy global del modelo base es del 93.33% con un F1-score macro de 0.9333, lo que indica un rendimiento muy sólido para un modelo inicial sin optimización de hiperparametros.

6. Comparación: Modelo Base vs Modelo Optimizado

Esta sección documenta el segundo aporte personal al notebook. Se agrego una visualización de barras que compara directamente el accuracy del modelo base contra el modelo obtenido mediante búsqueda de hiperparametros con RandomizedSearchCV.

6.1 Proceso de optimización de Hiperparametros

Para mejorar el modelo base se implementaron dos estrategias de búsqueda de hiperparametros utilizando la librería scikeras para integrar Keras con scikit-learn:

Método	Estrategia	Combinaciones	Mejor Accuracy
GridSearchCV	Prueba TODAS las combinaciones de la grilla	16 combinaciones	90.00%
RandomizedSearchCV	Prueba N combinaciones ALEATORIAS del espacio	10 combinaciones	95.83%

6.2 Mejores Hiperparametros Encontrados

Hiperparametro	Mejor valor (Random Search)
n_neurons	64
n_hidden	1
learning_rate	0.01
epochs	20
batch_size	32

6.3 Resultado del Modelo Optimizado

Entrenando el mejor modelo encontrado con los hiperparametros óptimos se obtuvieron los siguientes resultados en el conjunto de validación:

Clase	Precision	Recall	F1-Score	Support
setosa	1.00	1.00	1.00	10
versicolor	1.00	0.90	0.95	10
virginica	0.91	1.00	0.95	10
macro avg	0.97	0.97	0.97	30

6.4 Comparación Final

El grafico de barras agregado al notebook permite visualizar de forma inmediata el impacto de la optimización:

Modelo	Accuracy	F1-Score Macro	Errores en Validacion
Modelo Base (manual)	93.33%	0.9333	2 de 30
Modelo Optimizado (Random Search)	96.67%	0.9667	1 de 30

Codigo agregado

```
modelos = ['Modelo Base\n(manual)', 'Mejor Modelo\n(Random Search)']
accuracies = [0.9333, 0.9667]
colores = ['steelblue', 'orange']
```

```
plt.figure(figsize=(6, 4))
barras = plt.bar(modelos, accuracies, color=colores, width=0.4)
plt.ylim(0.85, 1.0)
plt.title('Comparacion de Accuracy: Base vs Optimizado')
plt.ylabel('Accuracy en Validacion')
```

```
for barra, acc in zip(barras, accuracies):
    plt.text(barra.get_x() + barra.get_width()/2,
```

acc + 0.003, f'{acc:.2%}',
ha='center', fontweight='bold')
plt.tight_layout()
plt.show()

6.5 Análisis de la Mejora

- La búsqueda de hiperparametros redujo los errores de 2 a 1 en el conjunto de validación de 30 muestras.
- RandomizedSearchCV supero a GridSearchCV porque exploro un espacio de búsqueda mas amplio, incluyendo el learning_rate como variable de optimización.
- El mejor modelo usa solo 1 capa oculta con 64 neuronas (frente a 2 capas con 16 y 8 del modelo base), lo que demuestra que mas capas no siempre es mejor.
- El learning_rate optimo de 0.01 es 10 veces mayor al valor por defecto (0.001), permitiendo una convergencia más rápida en las 20 épocas disponibles.

7. Conclusiones

Este proyecto demostró con éxito el flujo completo de desarrollo de una Red Neuronal Artificial para clasificación multiclase:

- El modelo base alcanzo un accuracy del 93.33% con una arquitectura simple de 243 parametros, demostrando que las redes neuronales son efectivas incluso en datasets pequeños.
- La estandarización de variables (StandardScaler) es un paso critico: sin ella, las redes neuronales convergen más lentamente o no convergen.
- La búsqueda sistemática de hiperparametros (RandomizedSearchCV) mejoro el accuracy al 96.67%, reduciendo los errores a la mitad con solo 10 iteraciones de búsqueda.
- Las visualizaciones agregadas (matriz de confusion y comparacion de modelos) facilitan enormemente la comunicación de los resultados a audiencias técnicas y no técnicas.
- La confusión entre versicolor y virginica es un resultado esperado y documentado en la literatura, validando que el modelo funciona correctamente.

Los aportes personales de las secciones 13 y 14 complementan el notebook original agregando valor analítico y visual al trabajo, facilitando la interpretación de los resultados y la comunicación de la mejora obtenida mediante la optimización de hiperparametros.